Chapitre 5 : Chaîne de caractères

Hassen NAKBI ::: hassen.nakbi@gmail.com

1ere année Licence

27 novembre 2023

Introduction

Chaîne de caractères

Une chaîne de caractères est une séquence ordonnée de caractères. Les caractères peuvent inclure des lettres, des chiffres, des espaces, des symboles, et même des caractères spéciaux tels que les retours à la ligne. Une chaîne est souvent utilisée pour représenter des paragraphes et du texte dans un programme.

2 Chaîne de caractères

- 2.1 Définition
- 2.2 Propriétés
- 2.3 Définition algorithmique
- 2.4 Chaîne en langage C
- 2.5 Opérations sur une chaîne

Définition algorithmique

En algorithmique, une chaîne de caractères est généralement représentée comme un tableau (ou une liste) de caractères. Chaque caractère occupe une position spécifique dans la séquence. Cependant, en langage C la chaîne est terminée par un caractère spécial appelé le "caractère nul" noté "\0". Ce caractère indique la fin de la chaîne.

2 Chaîne de caractères

- 2.1 Définition
- 2.2 Propriétés
- 2.3 Définition algorithmique
- 2.4 Chaîne en langage C
- 2.5 Opérations sur une chaîne

Propriétés

- La **taille** d'une chaîne représente la capacité maximale de la mémoire réservée pour une chaîne, tandis que la **longueur** d'une chaîne représente le nombre réel de caractères dans la chaîne.
- Lors de la définition initiale d'une chaîne, elle devient une chaîne **immuable**, et il est impossible de lui attribuer une nouvelle chaîne par la suite.
- Les caractères composant une chaîne peuvent être accédés par leur indice en utilisant l'opérateur d'indexation.

2 Chaîne de caractères

- 2.1 Définition
- 2.2 Propriétés
- 2.3 Définition algorithmique
- 2.4 Chaîne en langage C
- 2.5 Opérations sur une chaîne

Définition d'une chaîne

Définition algorithmique

```
Algorithme principal chaîne
CONST N = 50
TYPE chaine = tableau[1 .. N] de caractères
VAR CH : chaine
procédure Lirechaine (ST : chaine)
Début
Écrire("Saisir une chaine")
Lire(ST)
Fin
DÉBUT
Lirechaine(CH)
Écrire(CH, " de longueur : ",long(CH) )
FIN
```

2 Chaîne de caractères

- 2.1 Définition
- 2.2 Propriétés
- 2.3 Définition algorithmique
- 2.4 Chaîne en langage C
- 2.5 Opérations sur une chaîne

Fichier bibliothèque string.h

Le fichier d'en-tête string.h est une bibliothèque standard en langage C qui fournit des fonctions pour la manipulation de chaînes de caractères. Cette bibliothèque offre diverses fonctions pour effectuer des opérations courantes sur les chaînes, telles que la copie, la comparaison, et la concaténation.

Fichiers bibliothèques

- Fichier <stdio.h>

Fonction	**Description**
`puts(S)`	Affiche à l'écran la chaîne S avec un retour à la ligne.
`gets(S)`	Elle fait attendre au clavier une suite des caractères qui se termine par la touche clavier entrée.

- Fichier <string.h>

Fonction	**Description**
`strlen(S)`	Elle fait rendre la longueur réelle d'une chaîne S.

Chaîne en langage C

Programmation C

```
#include <stdio.h>
#include <string.h>
#define N 50
char CH[N];
void Lirechaine(char ST[ ]){
printf("Saisir une chaîne : ");
gets(ST);}
int main( ) {
Lirechaine(CH);
printf("%s de longueur %hi ",CH,strlen(CH));
return 0;}
```

Accès aux éléments d'une chaîne

Les caractères composant une chaîne peuvent être accédés par leur indice en utilisant l'opérateur d'indexation.

```
Algorithme principal chaîne
CONST N = 50
TYPE chaine = tableau[1 .. N] de caractères
VAR CH : chaine
DÉBUT
CH[1] ⟵ 'b' - 32
CH[2] ⟵ 'F' + 32
CH[3] ⟵ CH[1] + 4
CH[4] ⟵ 'a' + aléa(26)
FIN
```

```
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
#define N 50
char CH[N];
int main( ) {
CH[0] = toupper('b');
CH[1] = tolower('F');
CH[2] = CH[0] + 4;
CH[3] = 'a' + rand()%26;
return 0;}
```

2 Chaîne de caractères

- 2.1 Définition
- 2.2 Propriétés
- 2.3 Définition algorithmique
- 2.4 Chaîne en langage C
- 2.5 Opérations sur une chaîne

Opérations sur une chaîne

Opérations sur une chaîne

Les chaînes peuvent être manipulées de différentes manières, notamment la concaténation, la copie, la comparaison, la recherche de sous-chaînes, et l'extraction de sous-chaînes.

Fichier bibliothèque <string.h>

Fonction	Description
`strcpy(S1,S2)`	Copie le contenu de S2 dans la chaîne S1.
`strncpy(S1,S2,n)`	Copie les premiers n caractères de S2 dans S1.
`strcat(S1,S2)`	Concatène la chaîne S2 à la fin de S1.
`strncat(S1,S2,n)`	Concatène les premiers n caractères de chaîne S2 à la fin de S1.
`strcmp(S1,S2)`	Compare deux chaînes S1 et S2 selon les codes ASCII.
`strncmp(S1,S2,n)`	Compare les premiers n caractères de chaîne S2 et le contenu de S1 selon les codes ASCII.

a. Copie des chaînes

L'opération de copie de chaîne consiste à créer une copie identique d'une chaîne de caractères existante dans une nouvelle zone mémoire.

```
Algorithme principal copiechaîne
CONST N = 50
TYPE chaine = tableau[1 .. N] de caractères
VAR CH : chaine
DÉBUT
copie(CH,"Salut ")
FIN
```

a. Copie des chaînes

```
#include <stdio.h>
#include <string.h>
#define N 30
char CH[N];
int main( ) {
strcpy(CH,"Salut ");
return 0;}
```

b. Concaténation des chaînes

La concaténation est une opération qui consiste à fusionner deux chaînes de caractères pour en former une nouvelle. Elle est généralement réalisée en ajoutant les caractères de la deuxième chaîne à la fin de la première.

```
Algorithme principal concatchaîne
CONST N = 50
TYPE chaine = tableau[1 .. N] de caractères
VAR CH1, CH2 : chaine
Fin
DÉBUT
copie(CH1,"Salut ")
copie(CH2,"Mohamed")
CH1 ⟵ CH1 + CH2
FIN
```

b. Concaténation des chaînes

```
#include <stdio.h>
#include <string.h>
#define N 30
char CH1[N],CH2[N];
int main( ) {
strcpy(CH1,"Salut ");
strcpy(CH2,"Mohamed");
strcat(CH1,CH2);
return 0;}
```

c. Comparaison des chaînes

L'opération de comparaison de chaînes de caractères est utilisée pour déterminer si deux chaînes sont égales ou non.

Algorithme principal comparechaîne
CONST N = 50
TYPE chaine = tableau[1 .. N] de caractères
VAR CH1, CH2 : chaine
Fin
DÉBUT
copie(CH1,"Salut ")
copie(CH2,"Mohamed")
SI (**compare(CH1,CH2)** $\neq$ 0) alors
Écrire("Les deux mots sont différents")
FIN

c. Comparaison des chaînes

```
#include <stdio.h>
#include <string.h>
#define N 30
char CH1[N],CH2[N];
int main( ) {
strcpy(CH1,"Salut ");
strcpy(CH2,"Mohamed");
if (strcmp(CH1,CH2) != 0)
printf("Les deux mots sont différents");
return 0;}
```

d. Conversion des chaînes

En algorithmique, les fonctions de conversion sont utilisées pour transformer des données d'un type à un autre.
Convertir une chaîne de caractères vers un entier.

```
Algorithme principal convchaîne
CONST N = 50
TYPE chaine = tableau[1 .. N] de caractères
VAR CH : chaine
a :entier
DÉBUT
copie(CH,"123")
a ⟵ conventier(CH1)
FIN
```

d. Conversion des chaînes

```
#include <stdio.h>
#include <stdlib.h>
#define N 50
char CH[N];
short a;
int main( ) {
strcpy(CH,"123");
a = atoi(CH);
return 0;}
```

d. Conversion des chaînes

Convertir une chaîne de caractères vers un réel.

```
Algorithme principal convchaîne
CONST N = 50
TYPE chaine = tableau[1 .. N] de caractères
VAR CH : chaine
b : réel
DÉBUT
copie(CH,"7.25")
b ⟵ convréel(CH)
FIN
```

d. Conversion des chaînes

```
#include <stdio.h>
#include <stdlib.h>
#define N 50
char CH[N];
float b;
int main( ) {
strcpy(CH,"7.25");
b = atof(CH);
return 0;}
```

d. Conversion des chaînes

Convertir un entier ou un réel vers une chaîne.

```
Algorithme principal convchaîne
CONST N = 50
TYPE chaine = tableau[1 .. N] de caractères
VAR CH : chaine ; a :entier ; b : réel
DÉBUT
a ⟵ 7
b ⟵ 7/2
convch(CH,a)
copie(CH+1,"divisé par deux")
convch(CH+16,b)
FIN
```

d. Conversion des chaînes

```
#include <stdio.h>
#define N 50
char CH[N];
short a = 7;
float b = a / 2;
int main( ) {
sprintf(CH,"%hi divise par deux %f",a,b);
return 0;}
```

3 Tableau des chaînes

- 3.1 Définition
- 3.2 Propriétés
- 2.3 Définition algorithmique
- 3.4 Tableau des chaînes en langage C

Définition

En algorithmique, un tableau de chaînes est modélisé comme une matrice qui représente un ensemble de lignes de chaînes. Chaque ligne du tableau est une chaîne de caractères différente, et chaque colonne représente un caractère composant une chaîne.

3 Tableau des chaînes
- 3.1 Définition
- 3.2 Propriétés
- 2.3 Définition algorithmique
- 3.4 Tableau des chaînes en langage C

Propriétés

- **Taille fixe** : La taille du tableau des chaînes, c'est-à-dire le nombre de chaînes qu'il peut contenir, est généralement fixe à la définition.
- **Longueurs variables** : Chaque chaîne dans le tableau peut avoir une longueur différente. Cela permet de stocker des chaînes de caractères de longueurs variables.
- **Accès par ligne** : L'accès aux chaînes se fait généralement de manière séquentielle, en parcourant les lignes du tableau.

3 Tableau des chaînes
- 3.1 Définition
- 3.2 Propriétés
- 2.3 Définition algorithmique
- 3.4 Tableau des chaînes en langage C

Définition d'un tableau des chaîne

```
Algorithme principal tabchaîne
CONST M = 10, N = 50
TYPE chaine = tableau[1 .. N] de caractères
TYPE tabchaine = tableau[1 .. M][1 .. N] de chaines
VAR TCH : tabchaine
procédure scanst(ST : tabchaine)

VAR i : entier                          DÉBUT
Début                                   scanst(TCH)
Pour i de 1 à M pas ⟵ 1                 FIN
faire
Écrire("Saisir une chaine")
Lire(ST[i])
FinPour
Fin
```

3 Tableau des chaînes
- 3.1 Définition
- 3.2 Propriétés
- 2.3 Définition algorithmique
- 3.4 Tableau des chaînes en langage C

Tableau des chaînes en langage C

```
#include <stdio.h>
#include <string.h>
#define M 10
#define N 50
char TCH[M][N];
void scanst(char ST[ ][N]){
for(i=0;i<M;i++) {
printf("Saisir une chaîne : ");
gets(ST[i]);}}
int main( ) {
scanst(TCH);
return 0;}
```

a. Copie dans un tableau des chaînes

```
Algorithme principal tabchaînecopie
CONST M = 10, N = 50
TYPE chaine = tableau[1 .. N] de caractères
TYPE tabchaine = tableau[1 .. M][1 .. N] de chaines
VAR TCH : tabchaine
DÉBUT
copie(TCH[2],"Bonjour tout le monde")
FIN
```

a. Copie dans un tableau des chaînes

```
#include <stdio.h>
#include <string.h>
#define M 10
#define N 50
char TCH[M][N];
int main( ) {
strcpy(TCH[1],"Bonjour tout le monde");
return 0;}
```

b. Concaténation des chaînes

```
Algorithme principal concatchaîne
CONST M = 10, N = 50
TYPE chaine = tableau[1 .. N] de caractères
TYPE tabchaine = tableau[1 .. M][1 .. N] de chaines
VAR TCH : tabchaine
DÉBUT
copie(TCH[1],"Salut ")
copie(TCH[2],"Bonjour ")
TCH[1] ⟵ TCH[1] + "Ahmed"
TCH[2] ⟵ TCH[2] + "Mariem"
FIN
```

b. Concaténation des chaînes

```
#include <stdio.h>
#include <string.h>
#define M 10
#define N 50
char TCH[M][N];
int main( ) {
strcpy(TCH[0],"Salut ");
strcpy(TCH[1],"Bonjour ");
strcat(TCH[0],"Ahmed");
strcat(TCH[1],"Mariem");
return 0;}
```

c. Comparaison des chaînes

```
Algorithme principal comparechaîne
CONST M = 10, N = 50
TYPE chaine = tableau[1 .. N] de caractères
TYPE tabchaine = tableau[1 .. M][1 .. N] de chaines
VAR TCH : tabchaine
DÉBUT
copie(TCH[1],"Bonsoir")
copie(TCH[2],"Bonjour")
SI (compare(TCH[1],TCH[2]) ≠ 0) alors
Écrire("Les deux chaînes sont différentes")
FIN
```

c. Comparaison des chaînes

```
#include <stdio.h>
#include <string.h>
#define M 10
#define N 50
char TCH[M][N];
int main( ) {
strcpy(TCH[0],"Bonsoir");
strcpy(TCH[1],"Bonjour");
if (strcmp(TCH[0],TCH[1]) != 0)
printf("Les deux chaînes sont différentes");
return 0;}
```